

Código Fonte Python

Documento gerado para armazenamento de script de gestão hídrica.

Arquivo: script_bairros.py

```
import matplotlib.pyplot as plt
import numpy as np

def main():
    # ----- Entrada de dados -----
    n = int(input("Número de bairros: "))
    bairros = []
    for i in range(n):
        print(f"\n--- Bairro {i+1} ---")
        nome = input("Nome do bairro: ")
        demanda = float(input("Demanda diária (m³): "))
        capacidade = float(input("Capacidade do reservatório (m³): "))
        vol_inicial = float(input("Volume inicial disponível (m³): "))
        perc_perda = float(input("Percentual de perda na tubulação (%): "))
        custo_bomb = float(input("Custo de bombeamento por m³ (R$): "))
        bairros.append({
            'nome': nome,
            'demanda': demanda,
            'capacidade': capacidade,
            'vol_inicial': vol_inicial,
            'perc_perda': perc_perda,
            'custo_bomb': custo_bomb
        })

    # ----- Processamento -----
    resultados = []
    for b in bairros:
        # volume bombeado necessário para atender demanda considerando perda
        bombeado = b['demanda'] / (1 - b['perc_perda'] / 100)
        vol_entregue = b['demanda'] # efetivamente entregue (demanda atendida)
        vol_perdido = bombeado - vol_entregue # perdido na tubulação
        custo_diario = bombeado * b['custo_bomb'] # custo diário de bombeamento
        autonomia = b['vol_inicial'] / bombeado # dias de autonomia

        # classificação de risco
        if autonomia > 3:
            risco = "Baixo"
        elif 1 <= autonomia <= 3:
            risco = "Médio"
        else:
            risco = "Alto"

        resultados.append({
            'nome': b['nome'],
            'demanda': b['demanda'],
            'vol_entregue': vol_entregue,
            'vol_perdido': vol_perdido,
            'custo_diario': custo_diario,
            'autonomia': autonomia,
            'risco': risco
        })
```

```

# ----- Tabela -----
print("\n" + "="*80)
print(f"{'Bairro':<15} {'Demanda':>8} {'Entregue':>8} {'Perdido':>8}
{'Custo(R$)':>10} {'Autonomia(d)':>12} {'Risco':>8}")
print("-"*80)
for r in resultados:
    print(f"{r['nome']:<15} {r['demanda']:>8.1f}
{r['vol_entregue']:>8.1f} {r['vol_perdido']:>8.2f}
{r['custo_diario']:>10.2f} {r['autonomia']:>12.2f} {r['risco']:>8}")
print("="*80)

# ----- Identificação do bairro mais crítico ----- # critério:
maior risco (Alto > Médio > Baixo) e menor autonomia em caso de empate
ordem_risco = {'Alto': 0, 'Médio': 1, 'Baixo': 2}
critico = min(resultados, key=lambda r: (ordem_risco[r['risco']],
r['autonomia']))
print(f"\nBairro mais crítico: {critico['nome']} (Risco
{critico['risco']}, Autonomia {critico['autonomia']:.2f} dias)")

# ----- Gráficos -----
nomes = [r['nome'] for r in resultados]
demandas = [r['demanda'] for r in resultados]
entregues = [r['vol_entregue'] for r in resultados]
custos = [r['custo_diario'] for r in resultados]
riscos = [r['risco'] for r in resultados]

# 1) Gráfico de barras: Demanda vs Volume Entregue
fig, ax = plt.subplots(figsize=(10, 5))
x = np.arange(len(nomes))
largura = 0.35
ax.bar(x - largura/2, demandas, largura, label='Demanda',
color='steelblue')
ax.bar(x + largura/2, entregues, largura, label='Volume Entregue',
color='orange')
ax.set_xticks(x)
ax.set_xticklabels(nomes, rotation=45, ha='right')
ax.set_ylabel('Volume (m³)')
ax.set_title('Comparação: Demanda vs Volume Entregue')
ax.legend()
plt.tight_layout()
plt.show()

# 2) Gráfico de barras: Custo Diário por Bairro
fig, ax = plt.subplots(figsize=(10, 5))
cores_custo = ['#1f77b4' if c == min(custos) else '#d62728' for c in
custos] # destaca menor custo
ax.bar(nomes, custos, color=cores_custo)
ax.set_ylabel('Custo Diário (R$)')
ax.set_title('Custo de Bombeamento por Bairro')
ax.set_xticklabels(nomes, rotation=45, ha='right')
plt.tight_layout()
plt.show()

# 3) Gráfico de pizza: Distribuição dos Níveis de Risco
contagem = {'Baixo': riscos.count('Baixo'),
'Médio': riscos.count('Médio'),
'Alto': riscos.count('Alto')}
labels = list(contagem.keys())
sizes = list(contagem.values())

```

```
cores_pizza = ['#2ca02c', '#ffbb78', '#d62728']
fig, ax = plt.subplots()
ax.pie(sizes, labels=labels, autopct='%1.1f%%', colors=cores_pizza,
startangle=90)
ax.set_title('Distribuição dos Níveis de Risco')
plt.tight_layout()
plt.show()

if __name__ == "__main__":
    main()
```